Vulnerability Assessment Report

Target Website:OWASP juice shop(http://juice-shop.herokuapp.com/#)

Target IP: 54.220.192.176

Assessment Type: Read-Only Web Security Assessment

Prepared by: Ledet Tamiru

Program: Cyber Security Task 1 (2026) – Future Interns

Date:Feb-25-2026--Mar-25-2026

Table of Contents

1.1 Executive summary.....3
Methodology.....3
2.1 Assessment Scope.....3
2.2 Assessment Methodology.....4
2.3 Reconnaissance and Network Scanning.....4
3 Finding.....5
3.1 Vulnerability Summary.....5
3.2 Detail Vulnerability Findings :.....5
3.2.1 Content Security Policy (CSP) Header Not Set.....5
3.2.2 Cross-Domain Mis-configuration.....6
3.2.3 Vulnerable JavaScript Library.....7
3.2.4 Strict-Transport-Security Header Not Set.....8
3.2.5 Server Leaks Version Information via "Server" HTTP Header.....9
3.2.6 X-Content-Type-Options Header Missing.....10
4. Recommendations & Remediation.....11
4.1 Recommendation Prioritization.....11
5. Conclusion.....11

1.1 Executive summary

A vulnerability assessment of **OWASP Juice Shop** was conducted using a **passive, read-only approach**. No active exploitation was performed. Tools including **OWASP ZAP** and **Nmap** were used for passive scanning and reconnaissance, and browser developer tools were used to inspect HTTP responses, cookies, and security headers. The assessment identified **24 vulnerabilities**, such as missing security headers, insecure cookies, session management issues, and information disclosure, which may increase the risk of attacks like **Cross-Site Scripting (XSS)**. Addressing these issues will improve the application's overall security posture.

Total Alerts Record	Medium Risk level	Low Risk Level	Informational risk level
24	5	6	13

Methodology

2.1 Assessment Scope

The vulnerability assessment of **OWASP Juice Shop** aimed to identify security weaknesses using a **passive, read-only approach** without actively exploiting vulnerabilities. The analysis focused on publicly accessible components, including HTTP responses, security headers, cookies, and client-side configurations. All testing was conducted carefully to avoid affecting the normal operation of the application.

In Scope

- Passive analysis of HTTP responses and headers
- Cookie security configuration inspection
- Identification of information disclosure issues

Out of Scope

- Active exploitation of vulnerabilities
- Denial-of-Service (DoS) or brute-force attacks
- Any activity that could disrupt the application

2.2 Assessment Methodology

The assessment used a **passive, read-only approach** to identify security weaknesses without affecting the application.

Key activities included:

- **Network & Port Analysis:** Nmap was used to detect open ports and running services.
- **Passive Vulnerability Detection:** OWASP ZAP scanned HTTP requests/responses to find missing headers, insecure cookies, session issues, and information disclosure.
- **Manual Verification:** Browser Developer Tools confirmed findings and collected evidence.

Tools Used:

- **Nmap (Zenmap GUI)** – network and service reconnaissance
- **OWASP ZAP (Passive Scan)** – automated passive vulnerability detection
- **Browser Developer Tools** – manual verification and evidence collection

2.3 Reconnaissance and Network Scanning

Network reconnaissance was performed using **Nmap** to identify open ports and services running on the target system. This step helps determine the exposed services and potential attack surface of the application.

The scan results showed the following open ports:

Port	State	Service
80	Open	HTTP
443	Open	HTTPS

These open ports confirm that the target system is hosting a web application accessible through both HTTP and HTTPS protocols.

Command

nmap -sV -sC 54.220.192.176

```
└─$ nmap -sV -sC juice-shop.herokuapp.com
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-05 03:00 EST
Nmap scan report for juice-shop.herokuapp.com (54.220.192.176)
Host is up (0.0078s latency).
Other addresses for juice-shop.herokuapp.com (not scanned): 54.73.53.134 46.137.15.86
rDNS record for 54.220.192.176: ec2-54-220-192-176.eu-west-1.compute.amazonaws.com
Not shown: 955 closed tcp ports (reset), 43 filtered tcp ports (no-response)
PORT    STATE SERVICE    VERSION
80/tcp  open  tcpwrapped
|_http-cors: HEAD GET POST PATCH
443/tcp open  tcpwrapped
| ssl-cert: Subject: commonName=*.herokuapp.com
| Subject Alternative Name: DNS:*.herokuapp.com
| Not valid before: 2026-01-01T00:00:00
|_Not valid after:  2027-01-29T23:59:59
|_http-title: OWASP Juice Shop
|_http-cors: GET POST

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 29.48 seconds
```

3 Finding

3.1 Vulnerability Summary

Risk level	Number of Findings
Medium	
Low	

3.2 Detail Vulnerability Findings :

3.2.1 Content Security Policy (CSP) Header Not Set

Risk Level: Medium

Description
The application does not include a **Content Security Policy (CSP)** HTTP response header. CSP is a security mechanism that helps prevent attacks such as Cross-Site Scripting (XSS) and data injection by defining which sources of content are allowed to be loaded by the browser.

Impact
Without a CSP header, the application is more vulnerable to client-side attacks such as:

- Cross-Site Scripting (XSS)
- Malicious script injection
- Data theft through injected scripts

Attackers may exploit this weakness to execute unauthorized scripts in users' browsers.

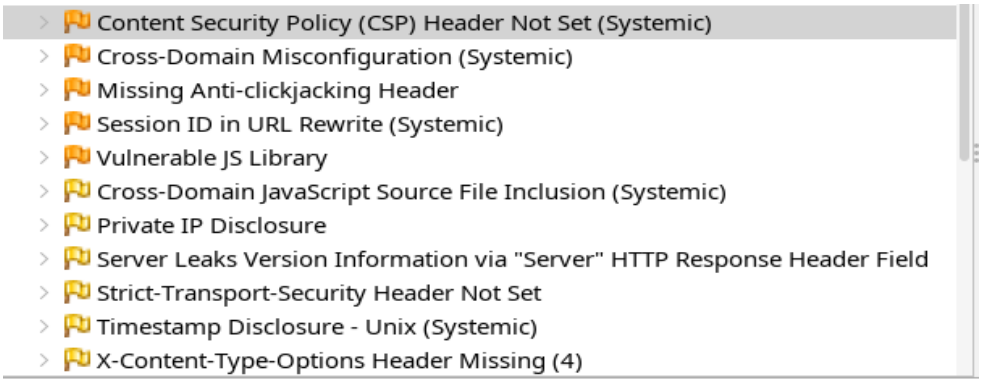
Recommendation
Configure the web server to include a **Content Security Policy** header that restricts the sources of scripts, styles, and other resources.

Example:

```
Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'
```

This helps ensure that only trusted sources can load content in the browser.

Proof of Concept



3.2.2 Cross-Domain Mis-configuration

Risk Level: Medium

Description
The application allows cross-domain access through misconfigured **Cross-Origin Resource Sharing (CORS)** settings. Improper CORS configurations can allow unauthorized domains to interact with the application.

Impact

Attackers may exploit this vulnerability to:

- Access sensitive information from another domain
- Perform unauthorized actions on behalf of authenticated users
- Bypass security restrictions between domains

Recommendation

Configure CORS policies properly by restricting allowed origins.

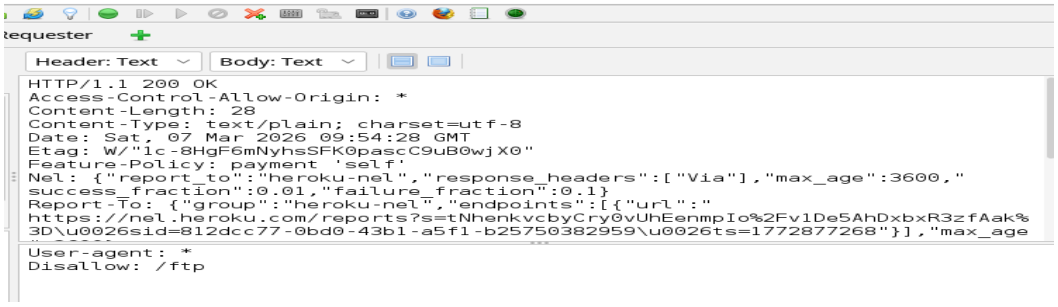
Example:

```
Access-Control-Allow-Origin: https://trusted-domain.com
```

Avoid using:

```
Access-Control-Allow-Origin: *
```

because it allows access from any domain.



3.2.3 Vulnerable JavaScript Library

Risk Level: Medium

Description

The application uses an **outdated or vulnerable JavaScript library**. Older versions of JavaScript libraries often contain known security vulnerabilities that attackers can exploit.

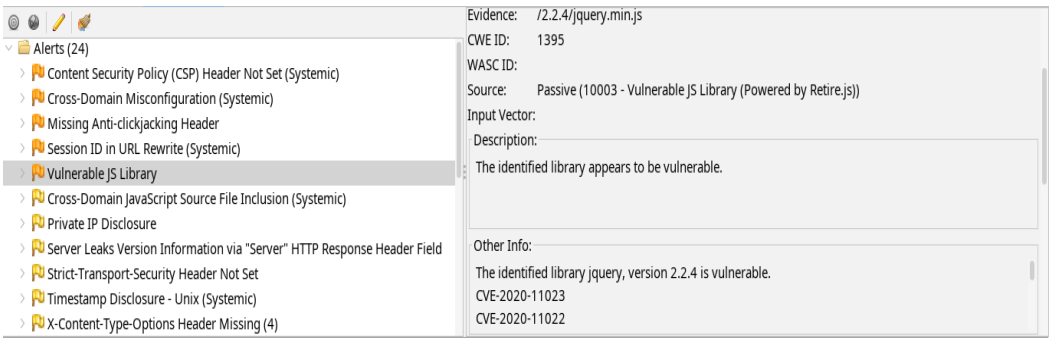
Impact : An attacker may exploit vulnerabilities in outdated libraries to:

- Execute malicious scripts
- Bypass security controls
- Perform Cross-Site Scripting attacks

Recommendation

- Identify the outdated JavaScript library
- Update it to the **latest secure version**
- Regularly review and patch third-party dependencies

Developers should maintain a **dependency management process** to keep libraries updated.



3.2.4 Strict-Transport-Security Header Not Set

Risk Level: Medium

Description

The application does not implement the **HTTP Strict Transport Security (HSTS)** header. HSTS ensures that browsers always communicate with the server using HTTPS instead of HTTP.

Impact

Without HSTS, attackers may perform:

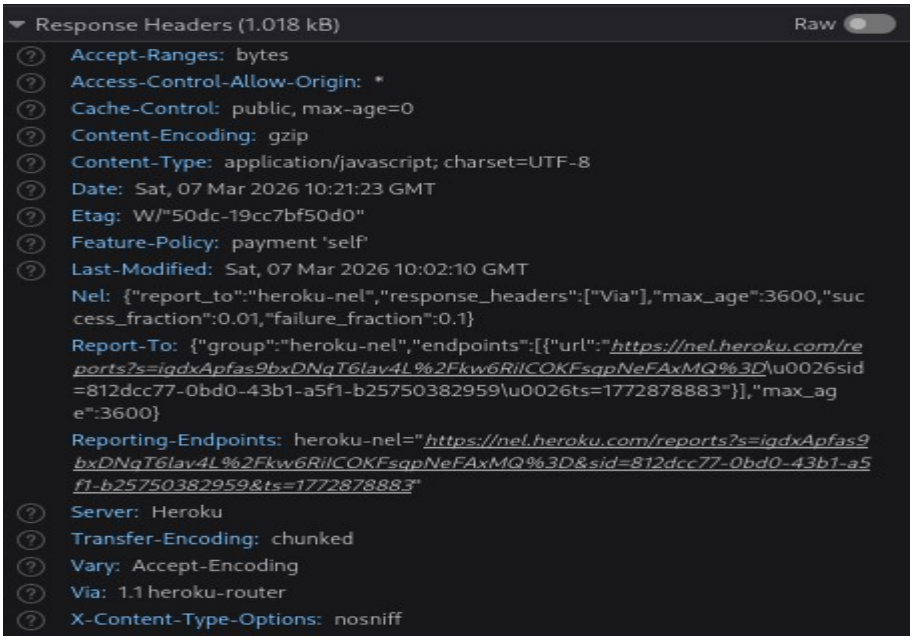
- SSL stripping attacks
- Man-in-the-Middle attacks
- Downgrade attacks to insecure HTTP
- **Recommendation**

Enable the HSTS header on the server.

Example:

Strict-Transport-Security: max-age=31536000; includeSubDomains
This forces browsers to always use HTTPS connections.

Screenshot of the response headers shows that the Strict-Transport-Security header is **not present**, meaning the browser is not instructed to always use HTTPS.



3.2.5 Server Leaks Version Information via "Server" HTTP Header

Risk Level: Low

Description

The server discloses its software platform through the Server HTTP header (Heroku). While no version number is exposed, revealing the server type can still help attackers identify potential vulnerabilities or configuration details.

Impact

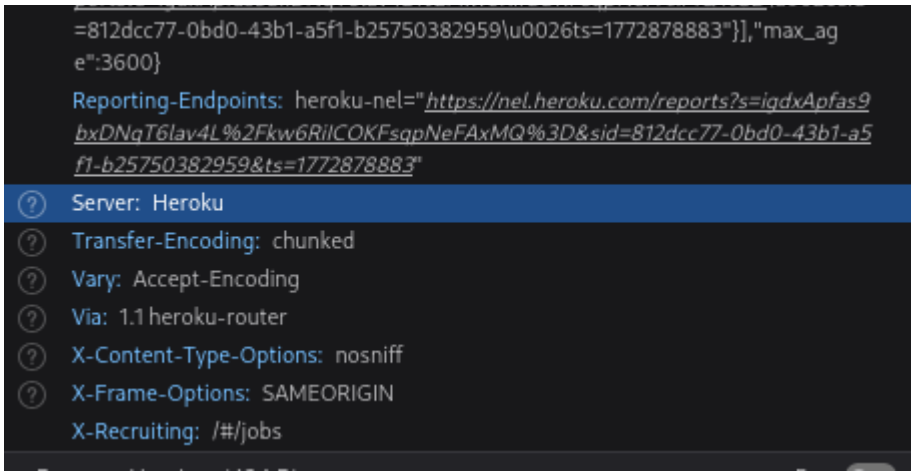
Attackers can use this information to:

- Attackers know the server platform (Heroku)
- They may look for **generic Heroku misconfigurations** or known platform issues
- Can inform targeted attacks, though risk is low

Recommendation

Hide the Server header if possible, or use generic values to reduce information exposure.

Example: Configure server to **remove or anonymize the header**.



3.2.6 X-Content-Type-Options Header Missing

Risk Level: Medium

Description:
The application does not include the X-Content-Type-Options: nosniff HTTP response header. This header instructs browsers **not to guess (MIME-sniff) the content type of files** and to

strictly follow the type declared by the server. Without it, browsers may interpret a file as a different type than intended, which can allow attackers to execute malicious scripts.

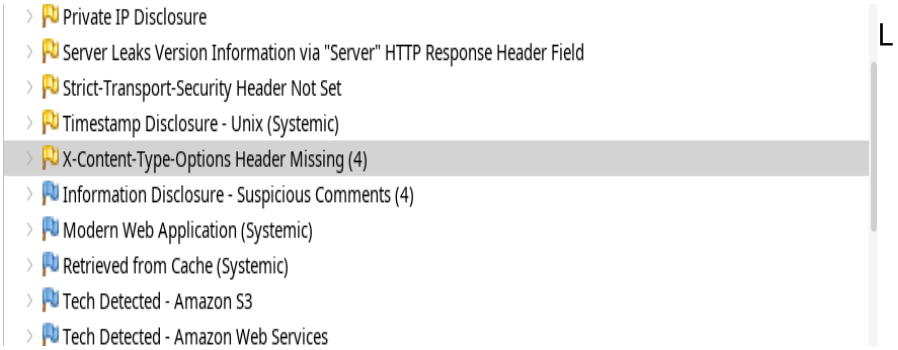
Impact:
If this header is missing, attackers may:

- Trick the browser into executing malicious scripts disguised as other file types (e.g., .txt, .js, .html).
- Bypass content-type checks and perform client-side attacks, including cross-site scripting (XSS) or data injection.

Recommendation:
Add the following HTTP header to all server responses:

```
X-Content-Type-Options: nosniff
```

This ensures that browsers **respect the declared content type** and reduces the risk of executing unexpected or harmful content.



4. Recommendations & Remediation

4.1 Recommendation Prioritization

Based on the findings from the assessment of **OWASP Juice Shop**, remediation should be prioritized according to risk severity.

Medium Priority

- Implement missing security headers such as **CSP** and **HSTS**.

- Fix insecure cookie configurations.
- Properly configure CORS policies.

Low / Informational

- Remove unnecessary server information from HTTP headers.
- Regularly update third-party libraries and dependencies.

Applying these recommendations will help reduce security risks and strengthen the application's overall security.

5. Conclusion

The vulnerability assessment of **OWASP Juice Shop** identified **24 security issues** using a passive, read-only approach. These include missing security headers, insecure cookie configurations, and information disclosure, which may increase the risk of attacks such as **Cross-Site Scripting (XSS)**. Implementing the recommended fixes will improve the application's security posture and reduce potential attack risks.